Advanced Day 4 — Session 4 (Hours 13–16)

Python Programming (Advanced) • HTTP Clients, Security Habits, Capstone Planning, and Checkpoint 2 (Persistence-Ready Core)

Session 4 Overview

Topics Covered Today

- Hour 13: HTTP client work — `requests` + JSON contracts
- Hour 14: Security basics — environment variables, safe secrets habits, and hashing concepts
- Hour 15: Capstone planning workshop — scope, milestones, and delivery path
- Hour 16: Checkpoint 2 — persistence-ready core + JSON save/load

Source Alignment

- `Advanced/lessons/lecture/Day4_Hour1_Advanced.md` → `# Day 4, Hour 1: HTTP Client Work – requests + JSON Contracts`
- `Advanced/lessons/lecture/Day4_Hour2_Advanced.md` → `# Day 4, Hour 2: Security`

Session 4 Outcomes

- Build a reliable HTTP client that handles timeouts, 4xx, 5xx, and invalid JSON
- Keep secrets out of source code using environment variables and `.env.example`
- Lock a realistic capstone MVP and choose a delivery path before building further
- Deliver a persistence-ready core that survives restart with logging and safe save/load

Scope Guardrails for Today

In Scope

- HTTP `GET` requests — timeout, `raise_for_status()`, JSON contract checks
- Environment-variable secret hygiene and hashing concept demo
- Capstone MVP scope — milestones, delivery path, README skeleton
- JSON persistence — safe-save pattern, `to_dict` / `from_dict`, file logging

Not Yet

- Full OAuth / JWT / production authentication systems
- Flask or other API server frameworks
- Enterprise schema validation tooling
- Database integration beyond a persistence-ready architecture

Hour 13: HTTP Client Work + JSON Contracts

Learning Outcomes

- Explain the difference between an HTTP client and an API server
- Send a `GET` request with a timeout using `requests`
- Use `raise_for_status()` to surface HTTP errors cleanly
- Validate required JSON keys and types before using the data
- Write a wrapper function so request logic lives in one place

The Mental Shift: Outside Data Is Untrusted

- Up to now, most data has lived inside the program
- Today the program talks to a service outside itself
- Outside data may be late, missing, malformed, or structurally unexpected
- Professional code behaves predictably when the world gets messy

Framing Rule

Outside data is untrusted until we check it.

> Source: `Advanced/lessons/lecture/Day4_Hour1_Advanced.md` → `## Section 1: Recap & Transition from Session 3`

HTTP Client Fundamentals

- A client sends a request; a server answers
- Request parts: method, URL, optional params / headers / body
- Response parts: status code, headers, body (often JSON)
- `GET` — give me data; `POST` — accept this data or create something

```
import requests

response = requests.get("https://example.com/api/items", timeout=5)
print(response.status_code)  # 200, 404, 500, etc.
print(response.text)          # raw body
```

This is approachable — but not yet robust. Timeout, status checks, and JSON validation still needed.

> Source: `Advanced/lessons/lecture/Day4_Hour1_Advanced.md` → `## Section 2:`

Timeout and `raise_for_status()`

```
# Single overall timeout — never hang forever
requests.get(url, timeout=5)

# Separate connect + read timeout
requests.get(url, timeout=(3, 10))
```

```
response = requests.get(url, timeout=5)
response.raise_for_status()   # HTTPError on 4xx / 5xx
data = response.json()
```

- Without a timeout, a slow server freezes the program indefinitely
- `raise_for_status()` keeps happy-path code clean and pushes failures into exception handlers

> Source: `Advanced/lessons/lecture/Day4_Hour1_Advanced.md` → `## Section 3: JSON Contracts, Timeouts, and Status Codes`

JSON Is a Contract

- Beginners assume the response matches exactly what they imagined
- APIs change; fields go missing; types drift from `int` to `str`
- Define the minimum contract required before trusting the data

```python
class APIContractError(Exception):
    """Raised when API data does not match the expected shape."""


def validate_post_contract(payload: dict) -> None:
    required: dict = {"id": int, "title": str, "body": str}
    for field, expected_type in required.items():
        if field not in payload:
            raise APIContractError(f"Missing required field: {field}")
        if not isinstance(payload[field], expected_type):
            raise APIContractError(
                f"'{field}' expected {expected_type.__name__}, "
                f"got {type(payload[field]).__name__}"
            )
```

Demo: Full Wrapper Function

```python
def fetch_post(post_id: int) -> dict | None:
    url = f"https://jsonplaceholder.typicode.com/posts/{post_id}"
    try:
        response = requests.get(url, timeout=(3, 10))
        response.raise_for_status()
        data = response.json()
        if not isinstance(data, dict):
            raise APIContractError("Expected a JSON object")
        validate_post_contract(data)
        return data
    except requests.Timeout:
        logger.error("The service timed out.")
    except requests.ConnectionError:
        logger.error("The service appears unreachable.")
    except requests.HTTPError as err:
        logger.error("HTTP error: %s", err.response.status_code)
    except requests.exceptions.JSONDecodeError:
        logger.error("The service returned invalid JSON.")
    except APIContractError as err:
        logger.error("JSON contract not satisfied: %s", err)
    return None
```

Friendly Failure Categories

- **Timeout** — "The service took too long to respond."
- **Connection problem** — "The service may be unreachable."
- **HTTP error** — "The service returned an error."
- **Invalid JSON** — "The service returned unreadable data."
- **Contract mismatch** — "The response did not match expectations."

Two Audiences, Two Channels

- User sees a calm, plain-language message
- Log file captures the full technical detail for developers

> Source: `Advanced/lessons/lecture/Day4_Hour1_Advanced.md` → `## Section 3: Friendly Failure Categories`

Lab: API Consumer (26 min)

Tasks

- Call an HTTP endpoint with `timeout=...` on every request
- Parse JSON and print selected fields in a user-friendly format
- Use `raise_for_status()` and validate required keys before downstream use
- Handle at least: timeout, connection failure, 404, 500, invalid JSON, contract mismatch
- Keep all request logic in one wrapper function — no copy/paste

Completion Criteria

- Happy path parses and prints cleanly
- Client never hangs indefinitely
- Each failure category produces a friendly message, not a raw traceback

Common Pitfalls (Hour 13)

- No timeout — program may hang forever waiting for a slow server
- Calling `.json()` before confirming the status code is acceptable
- Accessing `data["title"]` without first confirming `data` is a dictionary
- One broad `except Exception:` at the top masking the real error category
- Logging raw exception traces directly to users instead of developer logs
- Duplicating `requests.get(...)` logic in multiple places instead of a wrapper

Quick Check: Why is "the server returned valid JSON" not the same as "the response satisfies our contract"?

> Source: `Advanced/lessons/lecture/Day4_Hour1_Advanced.md` → `## Section 5: Common Pitfalls`

Hour 14: Security Basics — Secrets, Env Vars, and Hashing

Learning Outcomes

- Explain why secrets must not be hard-coded in source files or commits
- Load secrets from environment variables with a helper that fails early on missing values
- Create `.env.example` with placeholder values and update `.gitignore` accordingly
- Describe hashing vs encryption at a conceptual level and keep scope honest
- Gate an admin action on configuration rather than a hard-coded literal

Security Is About Habits, Not Heroics

- This hour prevents very common, very avoidable mistakes — not everything
- The scope boundary is intentional and important

Not This Hour

- OAuth, JWT, OIDC, or production identity systems
- Full cryptographic design

This Hour

- Where sensitive values live and how they move
- Environment variable loading pattern
- Concept-level hashing demonstration with honest scope labeling

> Source: `Advanced/lessons/lecture/Day4_Hour2_Advanced.md` → `## Section 1:`

Hard-Coded Secrets Are a Trap

```
# Do not do this
ADMIN_KEY = "super-secret-real-key"
```

Why This Fails in Practice

1. Secrets get committed to version control — deleting the line later leaves history
2. Secrets spread through screenshots, code reviews, and copy/paste
3. Rotation requires a code change and redeploy, not just a config update
4. Unsafe habits spread across the team

The Safe Pattern

```
import os

api_key = os.environ.get("APP_API_KEY")
```

`get_required_env()` Helper

```python
import os


def get_required_env(name: str) -> str:
    value = os.environ.get(name)
    if not value:
        raise RuntimeError(f"Missing required environment variable: {name}")
    return value
```

- Fails loudly when a required variable is absent
- Silently receiving `None` and stumbling later is harder to debug
- Use this in any admin or API-enabled script

> Source: `Advanced/lessons/lecture/Day4_Hour2_Advanced.md` → `## Section 2: Environment Variables as a Simple, Practical Pattern`

`.env.example` Template and `.gitignore`

```
# .env.example — safe to commit, placeholders only, never real values
APP_API_KEY=your-api-key-here
APP_ADMIN_KEY=replace-with-local-admin-key
APP_ENV=development
```

```
# .gitignore — exclude local secret files
.env
.env.local
.env.development
.env.production
!.env.example
secrets.json
```

- `.env.example` documents required names for collaborators
- `.env` is local configuration — never commit it

> Source: `Advanced/lessons/lecture/Day4_Hour2_Advanced.md` → `## Section 2:`

Hashing vs Encryption

- **Encryption** is designed to be reversible with the right key
- **Hashing** is one-way — no key "decrypts" a digest back to the original
- Common pattern: hash a new input, compare its digest to a stored digest

```python
import hashlib
import secrets


def sha256_digest(value: str) -> str:
    return hashlib.sha256(value.encode("utf-8")).hexdigest()


# Safe constant-time comparison (avoids timing side-channels)
match = secrets.compare_digest(
    sha256_digest(provided_key),
    sha256_digest(expected_key),
)
```

Demo: Environment Variable + Gated Admin Action

```python
import hashlib, logging, os, secrets

logger = logging.getLogger(__name__)
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")


def run_admin_action(provided_key: str) -> None:
    expected_key = get_required_env("APP_ADMIN_KEY")
    if not secrets.compare_digest(
        sha256_digest(provided_key),
        sha256_digest(expected_key),
    ):
        logger.warning("Admin action denied due to invalid key.")
        print("Admin action denied.")
        return
    logger.info("Admin action approved.")
    print("Admin action completed.")
```

- `get_required_env()` fails early if the variable is absent
- User sees "denied / completed"; log captures the event for audit

Lab: Secure-ish Configuration (18 min)

Tasks

- Read a required value from an environment variable using `get_required_env()`
- Gate one admin-only action on that value — no hard-coded literals anywhere
- Create `.env.example` with placeholder values only
- Update `.gitignore` to exclude `.env` and similar local files
- Add a README note explaining that env vars must be set before running admin actions
- Log whether the admin action was approved or denied

Completion Criteria

- No secret value appears in source code or commit history
- Missing env var fails clearly and early with a useful message

Common Pitfalls (Hour 14)

Security Habit Checklist — ask before every privileged action

1. Is this value sensitive or just general configuration?
2. Does it appear anywhere in source, screenshots, or commit history?
3. Can the value rotate tomorrow without a code change?
4. Are required variable names documented in README or `.env.example`?
5. Are local secret files excluded by `.gitignore`?

- Real secrets visible in demo or example files
- Committing `.env` because `.gitignore` was not updated in time
- Printing secret values to "prove the load worked" — use downstream behavior instead
- Treating hashing and encryption as interchangeable terms

Hour 15: Capstone Planning Workshop

Learning Outcomes

- Lock a realistic capstone MVP rather than an aspirational wish list
- Break the project into six milestones with explicit definitions of done
- Choose GUI-first or API-first and justify that choice in writing
- Create a README skeleton and a one-page plan before building further

What This Workshop Is Really For

- Replace vague enthusiasm with a concrete delivery plan
- A smaller finished MVP beats a large idea that never stabilizes
- Planning protects creativity from chaos — it does not shrink it

Driving Question

"What is the smallest version that still proves the core value?"

Opening Prompt

In one sentence: what should your capstone help a user do?

> Source: `Advanced/lessons/lecture/Day4_Hour3_Advanced.md` → `## Section 1: Reframe the Capstone`

Recommended Milestone Sequence

1. **Core** — business rules that work without a GUI
2. **Persistence** — state survives restart; corrupted storage is handled
3. **UI** — primary workflow through the chosen surface
4. **Analytics** — at least one meaningful derived insight from real data
5. **Tests** — repeatable checks for core rules and failure paths
6. **Package** — coherent layout, usable README, repeatable entry point

Do not jump to UI or analytics before the core and persistence are solid — a GUI only hides instability temporarily.

> Source: `Advanced/lessons/lecture/Day4_Hour3_Advanced.md` → `## Section 2: Milestones and Definition of Done`

Definition of Done for Each Milestone

Milestone	Done When
Core	Service logic works correctly in isolation
Persistence	Data survives restart; corrupted storage is handled
UI	User completes the primary workflow via chosen interface
Analytics	At least one useful insight is computed from project data
Tests	Key business rules and failure paths have repeatable checks
Package	Coherent layout, README, and repeatable run command exist

> Source: `Advanced/lessons/lecture/Day4_Hour3_Advanced.md` → `## Section 2: Milestones and Definition of Done`

Delivery Path: GUI-First vs API-First

GUI-First

- Best for human-operator tools, tracker dashboards, data-entry apps
- Tangible demo early; easy for non-technical audiences
- Risk: logic hides in callbacks; tests get postponed

API-First

- Best for automation backends, multi-client systems, integration projects
- Clear service boundaries; easier to test logic directly
- Risk: less visually exciting early; may under-document usage

Non-Negotiable for Either Path

Core logic must live in reusable services — never inside the interface layer.

Sample Project Structure and README Skeleton

```
capstone_project/
├── README.md           # Project goal, MVP, setup (placeholder), run (placeholder)
├── src/
│   └── task_tracker/
│       ├── __init__.py
│       ├── models.py       # domain data
│       ├── services.py     # business rules
│       ├── persistence.py  # save/load
│       └── ui.py           # interface layer
├── tests/
│   └── test_services.py
└── data/
    └── sample_data.json
```

Write the README **now** — Project Goal, MVP Features, Planned Milestones, Setup / Run / Testing placeholders.

> Source: `Advanced/lessons/lecture/Day4_Hour3_Advanced.md` → `## Section 4:`

One-Page Plan Template + Workshop (26 min)

```
## Project Goal — what user problem does this solve?
## MVP Scope — Feature 1 / Feature 2 / Feature 3
## Data Model — main entity, important fields, relationships
## Delivery Path — GUI-first or API-first + one-sentence justification
## Persistence Plan — JSON save/load now; database-ready later
## Analytics Idea — one useful summary or derived insight
## Test Plan — core rules to test + error cases to test
## Risks — biggest scope risk / biggest technical risk
## Stretch Goals — preserved ambition, clearly not MVP
```

Mini-budget: goal (3 min) → model (4 min) → path (3 min) → persistence (4 min) → analytics (3 min) → tests (4 min) → README skeleton (3 min) → stretch/risks (2 min)

> Source: `Advanced/lessons/lecture/Day4_Hour3_Advanced.md` → `## Section 5: Hands-On Workshop`

Common Pitfalls (Hour 15)

- Scope is a wish list rather than a realistic MVP for the course timeline
- No plan for persistence — data loss is discovered late
- No plan for testing — bugs accumulate in silence
- UI chosen before core logic exists — building a shell around unstable logic
- README postponed until the final session
- Stretch goals mixed into MVP so everything becomes mandatory
- Plan contains no concrete verbs — "analytics dashboard" is not an action

Quick Check: Which milestone do students most often jump to too early, and what instability does that risk hiding?

> Source: `Advanced/lessons/lecture/Day4_Hour3_Advanced.md` → `## Section 5: Common Pitfalls`

Hour 16: Checkpoint 2 — Persistence-Ready Core + JSON Save/Load

Learning Outcomes

- Organize the application as a package under `src/` with clear responsibility layers
- Save state using a safe-save pattern — write temp file first, then replace atomically
- Load JSON back into application objects and restore state across runs
- Handle missing files, invalid JSON, and validation failures with custom exceptions
- Configure logging to a file so all persistence events are captured for debugging

What Checkpoint 2 Proves

- The core survives beyond one run — state persists reliably
- The `src/` package layout separates domain, service, and persistence concerns
- Save and load failures are handled intentionally — not silently swallowed
- Logging captures meaningful events: save, load, missing file, invalid data, failures
- Custom exceptions still communicate domain and persistence problems clearly

Grading Focus

Correctness, structure, error handling, and logging. A small clean solution scores higher than a large fragile one.

> Source: `Advanced/lessons/lecture/Day4_Hour4_Advanced.md` → `## Section 1: Checkpoint Framing and Grading Focus`

Persistence-Ready Package Layout

```
project_root/
├── demo_persistence.py     # clear demonstration entry point
├── logs/
│   └── app.log
├── data/
│   └── state.json
└── src/
    └── reading_tracker/
        ├── __init__.py
        ├── errors.py           # custom exceptions
        ├── logging_config.py   # file logging setup
        ├── models.py           # domain data + to_dict/from_dict
        ├── persistence.py      # JSON save/load
        └── services.py         # business rules
```

Future database integration only requires changing `persistence.py` — the rest stays the same.

Domain Model with Serialization

```python
from dataclasses import asdict, dataclass, field


@dataclass(slots=True)
class Book:
    title: str
    author: str
    pages_read: int = 0


@dataclass(slots=True)
class ReadingState:
    books: list[Book] = field(default_factory=list)

    def to_dict(self) -> dict:
        return {"books": [asdict(book) for book in self.books]}

    @classmethod
    def from_dict(cls, data: dict) -> "ReadingState":
        return cls(books=[Book(**b) for b in data.get("books", [])])
```

Custom Exceptions + File Logging

```python
class AppError(Exception):
    """Base application error."""

class ValidationError(AppError):
    """Raised when domain validation fails."""

class PersistenceError(AppError):
    """Raised when save/load operations fail."""
```

```python
def configure_logging() -> None:
    log_dir = Path("logs")
    log_dir.mkdir(exist_ok=True)
    logging.basicConfig(
        filename=log_dir / "app.log",
        level=logging.INFO,
        format="%(asctime)s %(levelname)s %(name)s %(message)s",
    )
```

Safe-Save Pattern

```python
def save(self, state: ReadingState) -> None:
    self.path.parent.mkdir(parents=True, exist_ok=True)
    temp_path = self.path.with_suffix(self.path.suffix + ".tmp")
    try:
        with temp_path.open("w", encoding="utf-8") as f:
            json.dump(state.to_dict(), f, indent=2)
        temp_path.replace(self.path)          # atomic replacement
        logger.info("State saved to %s", self.path)
    except OSError as error:
        logger.exception("Failed to save state")
        raise PersistenceError("Could not save application state") from error
```

- Writing to `.tmp` first then replacing ensures the last good save is never partially overwritten
- A crash mid-write leaves the original file intact

> Source: `Advanced/lessons/lecture/Day4_Hour4_Advanced.md` → `## Section 3:`

Safe Load Pattern

```python
def load(self) -> ReadingState:
    if not self.path.exists():
        logger.info("State file missing; starting with default state")
        return ReadingState()
    try:
        with self.path.open("r", encoding="utf-8") as f:
            raw_data = json.load(f)
    except json.JSONDecodeError as error:
        logger.exception("State file contains invalid JSON")
        raise PersistenceError("State file is corrupted") from error
    except OSError as error:
        raise PersistenceError("Could not load application state") from error
    if not isinstance(raw_data, dict):
        raise PersistenceError("State file must be a JSON object")
    try:
        return ReadingState.from_dict(raw_data)
    except (TypeError, KeyError, ValueError) as error:
        logger.exception("State data failed validation")
        raise PersistenceError("State data structure was invalid") from error
```

Demo: Save → Restart → Load

```
store = JSONStateStore(Path("data/state.json"))
initial_state = store.load()            # missing file → default state
service = ReadingService(initial_state)
service.add_book(Book("Deep Work", "Cal Newport", pages_read=25))
service.add_book(Book("Clean Code", "Robert C. Martin", pages_read=10))
store.save(service.state)
print("State saved. Simulating restart...")

restarted_store = JSONStateStore(Path("data/state.json"))
reloaded = restarted_store.load()
restarted_service = ReadingService(reloaded)
for book in restarted_service.list_books():
    print(f"- {book.title} by {book.author} ({book.pages_read} pages read)")
```

The second instance reconstructs state from disk — that is the checkpoint proof.

> Source: `Advanced/lessons/lecture/Day4_Hour4_Advanced.md` → `## Section 4: Live Demo – Save -> Restart -> Load`

Checkpoint Evidence Checklist

Evidence	Check
State persists across runs	Save → restart → load shows same records
Missing file handled	First run produces default state, not a crash
Invalid JSON handled	`JSONDecodeError` caught; meaningful log entry present
Validation failure handled	Corrupt data raises `PersistenceError` cleanly
Logs are meaningful	Open `app.log` — save / load / failure entries visible
Separation of concerns	Name the one module that would change if JSON became a DB

> Source: `Advanced/lessons/lecture/Day4_Hour4_Advanced.md` → `## Section 5:`

Hands-On Checkpoint Build (26 min)

Mini-Budget

- 4 min — confirm `src/` package layout; identify model, service, persistence, errors, logging modules
- 4 min — add or review `to_dict()` / `from_dict()` so JSON receives plain dicts and lists
- 5 min — implement save/load with safe-save temp file and missing-file path
- 4 min — add file logging; verify save/load events produce meaningful entries
- 4 min — add `JSONDecodeError` and validation failure handling using custom exceptions
- 3 min — run save → restart → load; inspect the saved JSON file directly
- 2 min — write one reflection: what design decision would you change if starting over?

Common Pitfalls (Hour 16)

- Dumping raw Python objects to JSON without `to_dict()` serialization
- Overwriting the final file directly instead of writing to a temp file first
- Not catching `json.JSONDecodeError` specifically — bare `except:` hides the root cause
- Logging only to the console, not to a file
- Persistence code leaking into the service layer — save/load belongs in `persistence.py`
- Demoing a happy path without proving save → restart → load
- Ignoring missing-file behavior — first-run must work cleanly

Quick Check: If the app works in memory but loses all data after restart, which milestone is still incomplete?

Session 4 Wrap-Up

What We Built Today

- A reliable HTTP client wrapper with timeouts, status handling, and JSON contract checks
- Safer configuration using environment variables and `.env.example` hygiene
- A concrete one-page capstone plan with milestones, delivery path, and README skeleton
- A persistence-ready core that can save, restart, load, and log meaningful events

Key Session 4 Rule

Trustworthy software plans for failures, scope drift, and restart behavior before they happen.

Day 4 Homework / Study Checklist

- Re-run the HTTP wrapper with at least one real failure case triggered
- Remove any hard-coded secret from demo or capstone code
- Finalize the one-page capstone plan with stretch goals clearly separated
- Verify save → restart → load round-trip for the capstone core
- Confirm `app.log` contains meaningful save, load, and error entries

Source Alignment

- `Advanced/assignments/Advanced_Day4_homework.ipynb` → `## Reflection and Submission Checklist`

Next Session Preview

Session 5 (Hours 17–20)

- Tkinter GUI fundamentals — widgets, layout managers, and event loop
- Connecting a service layer to a GUI without mixing business logic into callbacks
- Building a GUI-first workflow for the capstone tracker
- Testing service logic independent of the interface layer

Thank You!

Validate outside data.

Keep secrets out of source.

Plan the MVP before building.

Prove persistence before expanding the interface.